

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
Занятие №11

по теме:
РЕАЛИЗАЦИЯ ОДНОВРЕМЕННОЙ ПЕРЕДАЧИ И ПРИЕМА НА ДВУХ
SDR. РЕАЛИЗАЦИЯ ФУНКЦИЙ СИМВОЛЬНОЙ СИНХРОНИЗАЦИИ

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватели:
Лектор
Семинарист
Семинарист

Калачиков А.А
Ахнашев А.В
Попович И.А

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	3
1.1	Введение	3
1.2	Цель	3
2	ПРАКТИКА	4
2.1	timing error detection	4
2.2	Описание	4
2.2.1	Реализация	4
3	ВЫВОД	7

ВВЕДЕНИЕ

1.1 Введение

На прошлом занятии мы реализовали часть схемы Гарднера, отвечающую за подсчет timing error. Теперь необходимо реализовать другую часть схемы Гарднера - loop filter.

1.2 Цель

Реализовать loop filter для схемы Гарднера.

ПРАКТИКА

2.1 timing error detection

2.2 Описание

Loop-filter нужен для того, чтобы скорректировать timing error. Если попытаться представить t_{ed} в виде функции (что-то типа синуса), то loop filter должен сгладить эту функцию и сделать переходы между t_{ed} более плавными. Для нас это означает, что с loop filter подбор оптимального момента взятия отсчетов станет точнее.

2.2.1 Реализация

```
std::vector<int16_t> synchronizer::gardner(const
    std::vector<std::complex<double>> &samples, int const L)
{
    /*define start parameters*/
    double Kp = 4;
    double BnTs = 0.000005;

    double zeta = std::sqrt(2) / 2;
    double theta = (BnTs / 10) / (zeta + 1 / (4 * zeta));
    double den = (1 + 2 * zeta * theta + theta * theta) * Kp;

    double K1 = -4 * zeta * theta / den;
    double K2 = -4 * theta * theta / den;

    double p1 = 0;
    double p2 = 0;

    int offset = L / 2;
    double e = 0;

    int n = 0;

    std::vector<int16_t> offset_list;
```

```

int max_i = samples.size() % L == 0 ? samples.size() / L :
    samples.size() / L + 1;

for (int i = 0; i < max_i; ++i)
{
    n = offset + L * i;
    /*calculate e - timing error*/
    e = (samples[n + L].real() - samples[n].real()) *
        samples[n + L / 2].real() +
        (samples[n + L].imag() - samples[n].imag()) *
        samples[n + L / 2].imag();

    /*loop filter logic*/
    p1 = e * K1;
    p2 += p1 + e * K2;

    p2 -= std::floor(p2);

    offset = static_cast<int>(std::round(p2 * L));
    offset_list.push_back(offset);
}

return offset_list;
}

```

На Python визуализируем е, чтобы увидеть, что loop filter делает с нашим `ted`

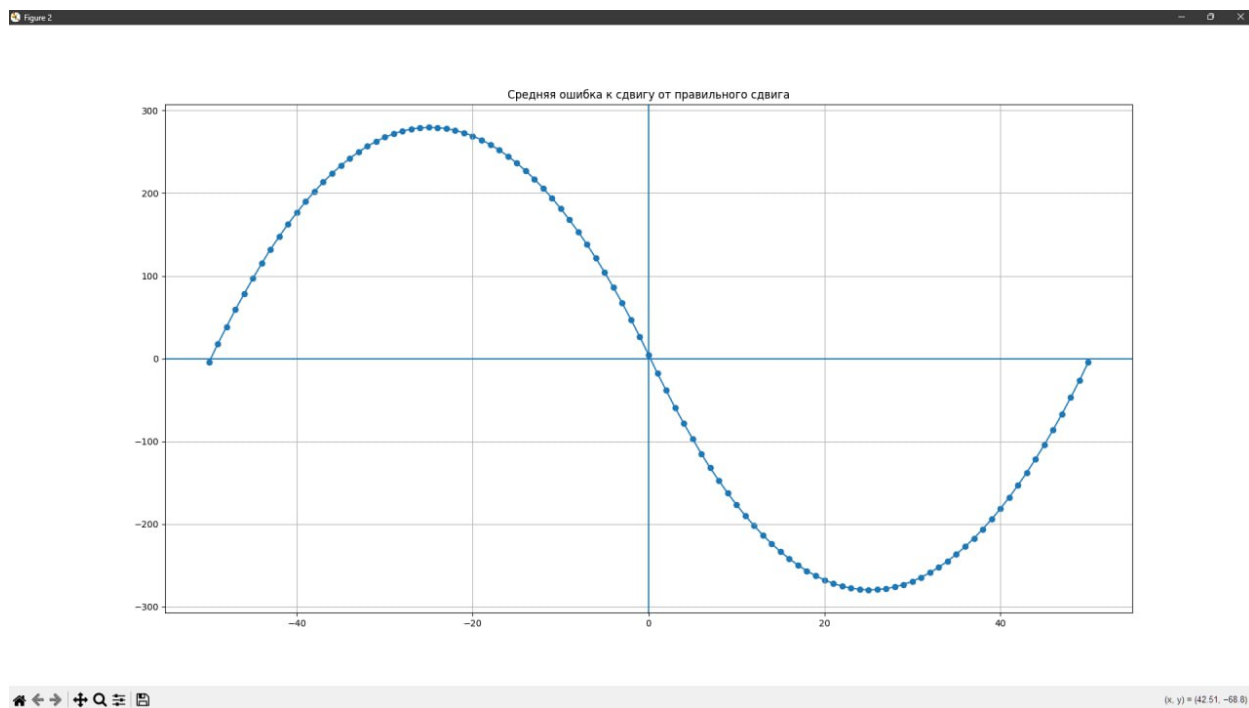


Рисунок 1 — TED

График значительно сгладился и стал больше похож на синус. Резкие скачки и пики исчезли. Такой ted будет наиболее точным

ВЫВОД

В ходе работы я на практике реализовал loop filter - одну из составных частей схемы Гарднера.